Programming Project 4: Monopoly Capstone

Team 1
CS 2430 Spring 2026

Johnathan McElprang, Keanu Cruz, Michael McCleary (Inactive), Pong Vodmongkol (Inactive)

Introduction

The purpose of this project is to create a simulation of the board game Monopoly (2018 Rules). In this simulation, we are interested in the empirical landing probability of each square. To do this, our team created a simulation engine that simulates a single player playing a game of Monopoly without tracking purchases, property, etc. Our focus was to hone in on a few key points of the game. How do the Chance and Community Chest cards affect landing data? How does the 3 double roll rule sending you to jail rule affect landing data? And, how does getting out of jail strategies affect landing data?

These components of the game matter most to our project because each of these components plays a role in how a player moves around the board. By simulating and examining these components, we are able to simulate large turn counts across several different game scenarios to justify our landing data. More specifically, we want to use these movement mechanics to see how our probabilities change between two specific get out of jail strategies. In this report, we will assume that if you have a get out of jail free card, it is used immediately, and the turn can be played. If a player does not have a get out of jail free card, then the player has two options. Option A: Pay $50 and leave on their next turn (money is assumed to be available for this strategy). Or Option B: The player has 3 chances to roll doubles on their two dice. If by the third turn they have failed to roll a double roll, then they are released from jail on their next turn. By simulating this data, we are hoping to see if these two strategies have convergence or if one is clearly superior to the other.

Implementation

For this project, we decided to code our work in Java. This language was the most comfortable language for the team to work with to create this project.

We used a total of 10 classes to keep our coding efficient and contained to the various game structure components. Most of these classes serve the basic purpose of being an interactable component of the game.

We will start with Square.java, this class is used to create an individual square that has a name, a type, and a visit count. In this class, we also focus on what happens when a player lands on a Chance, Chest, or Go To Jail Square.

Board.java utilizes Square.java to create our gameboard. In this class, we initialize all 40 spaces of the Monopoly board with proper names and types. This class is what handles the position location of each square that can be landed on and references the closest railroad and utility locations for our Chance and Chest cards.

Our Card.java and Deck.java classes work similarly to Square and Board. Card is an individual card that is initialized by Deck. In the Card class, we also handle card effects that will be paired with Card types used by Deck. Deck.java creates a drawpile for both the Chance and Community chest decks. These decks have the ability to be discarded to ensure that the probability remains fair and will also reshuffle the deck when cards run out. Deck also uses Card's value function to change a player's position on the board based on the official cards list for Monopoly.

Player.java handles how the player interacts with our board. In this class, we track double roll counts, get out of jail free cards, whether a player is in jail or not, and the player's overall position on the board. The main purpose of this class is to track the player's status in order to make sure all game components run correctly in the simulation engine.

Our Simulation Engine class is where our actual game flow logic lives. In this class, we use our Dice.java class to roll two six-sided dice to move our player. At the end of a player's turn, a square will be "resolved," meaning that it will apply any card effects, landing counts, or send to jail effects. In this class, we also implement our two different get out of jail strategies. We use our Strategy.enum class to keep track of the data for strategies A and B. For this class, we created a Run All Simulations method that runs 10 different simulations that each track landing data for 1000, 10,000, 100,000, and 1,000,000 turns for each strategy.

For our last two classes, we have our Result Analyzer class and our Main class. Result Analyzer is a helper class used to print and organize our data for our application in Main. Main is where we use our Run All Simulations method from our Simulation Engine Class. In this class, we output all data for the 10 simulation runs for both strategies and summarize the final results at the end of the console output.

Verification Approach

To do our testing for this project, we created JUnit test suites. We designed tests for all of the 10 classes used in the Monopoly Simulation, including Board, Card, Deck, Dice, Main, Player, Result Analyzer, Simulation Engine, Square, and Strategy.

The goal of this testing was to ensure that our classes behaved properly based on the rules given for the simulation by the project. We also wanted to make sure that our simulation produced valid and repeatable results. To check this, we tested expected outputs, state changes, and interactions between the different classes to make sure the movement and jail mechanics worked properly.

For each class, we started by checking its base functionality. For example, we wanted to make sure the dice rolls were producing valid numbers and that the player position and jail status were being updated properly. We also wanted to make sure that the board was being built properly with all the required spaces.

After we confirmed that the base functionality was working properly, we tested edge cases. We tested what would happen if the deck ran out of cards and made sure the jail logic would not break turn flow. We also tested cards that would add additional movements or scenarios.

One of the most important parts of our verification process was making sure the classes interacted with each other properly. This included landing on a square that would actually trigger drawing card events, and that the player's state was actually being changed. We wanted to make sure that everything worked together seamlessly.

The last thing we did was run a full simulation to confirm that everything worked together and that we were getting valid, properly formatted outputs for the simulation using both strategy A and B.

Here is our test for Roll Validity of Dice:

```
public class DiceTest {

    // Roll Validity Tests

    @Test
    void rollIsWithinValidRange() {
        Dice dice = new Dice();

        for (int i = 0; i < 200; i++) {
            int result = dice.roll();
```

```
        assertTrue(result >= 2 && result <= 12,
                "Invalid dice sum: " + result);
        }
    }
```

Here are our jail state tests for the player:

```
 // Jail State Tests

    @Test
    void jailStateCanBeSetAndRead() {
        Player p = new Player();

        p.setIsInJail(true);

        assertTrue(p.isInJail());
    }

    @Test
    void jailTurnsIncrementCorrectly() {
        Player p = new Player();

        p.incrementJailTurns();
        p.incrementJailTurns();

        assertEquals(2, p.getJailTurns());
    }

    @Test
    void jailTurnsCanBeSetDirectly() {
        Player p = new Player();

        p.setJailTurns(3);

        assertEquals(3, p.getJailTurns());
    }
```

Here is our test to make sure the sim engine is running all simulations:

```
    @Test
    void runAllSimsCompletesWithoutError() {
        SimulationEngine engine = new SimulationEngine();

        assertDoesNotThrow(() -> {
```

```
        engine.runAllSims();
    }
```

Results

Since the results of this project are too large to be contained in this report, we chose to highlight a sample of our data set for a single simulation run for Strategy A and Strategy B. Below are two tables that show a sample of one of our simulation runs at 1 million turns. In this data, we include the square name from the game board, the type of square, the landing count for that square, and lastly, the percentage of landings on that square based on the turn count.

Strategy A | Sim 10 | Turns 1000000

NAME	TYPE	LAND COUNT	PERCENT
GO	GO	37138	3.71%
Mediterranean Ave	PROPERTY	25267	2.53%
Community Chest	CHEST	21849	2.18%
Baltic Ave	PROPERTY	25225	2.52%
Income Tax	TAX	27275	2.73%
Reading Railroad	RAILROAD	35631	3.56%
Oriental Ave	PROPERTY	26547	2.65%
Chance	CHANCE	9238	0.92%
Vermont Ave	PROPERTY	27338	2.73%
Connecticut Ave	PROPERTY	27419	2.74%
Jail	JAIL	69320	6.93%
St. Charles Place	PROPERTY	31971	3.20%
Electric Company	UTILITY	31287	3.13%
States Ave	PROPERTY	27788	2.78%
Virginia Ave	PROPERTY	28641	2.86%
Pennsylvania Railroad	RAILROAD	34963	3.50%
St. James Place	PROPERTY	33010	3.30%
Community Chest	CHEST	30423	3.04%
Tennessee Ave	PROPERTY	34802	3.48%
New York Ave	PROPERTY	36392	3.64%
Free Parking	FREE PARKING	33643	3.36%
Kentucky Ave	PROPERTY	33664	3.37%
Chance	CHANCE	10966	1.10%
Indiana Ave	PROPERTY	32116	3.21%
Illinois Ave	PROPERTY	37945	3.79%
B. & O. Railroad	RAILROAD	36722	3.67%
Atlantic Ave	PROPERTY	31631	3.16%
Ventnor Ave	PROPERTY	31618	3.16%
Water Works	UTILITY	33201	3.32%
Marvin Gardens	PROPERTY	30375	3.04%
Go To Jail	GOTOJAIL	0	0.00%
Pacific Ave	PROPERTY	31528	3.15%
North Carolina Ave	PROPERTY	31143	3.11%
Community Chest	CHEST	27375	2.74%
Pennsylvania Ave	PROPERTY	29389	2.94%
Short Line Railroad	RAILROAD	28773	2.88%
Chance	CHANCE	9010	0.90%
Park Place	PROPERTY	25431	2.54%
Luxury Tax	TAX	25753	2.58%
Boardwalk	PROPERTY	31385	3.14%

Top 5 Most Visited Squares

NAME	TYPE	LAND COUNT	PERCENT
Jail	JAIL	69320	6.93%
Illinois Ave	PROPERTY	37945	3.79%
GO	GO	37138	3.71%
B. & O. Railroad	RAILROAD	36722	3.67%
New York Ave	PROPERTY	36392	3.64%

Strategy B | Sim 10 | Turns 1000000

NAME	TYPE	LAND COUNT	PERCENT
GO	GO	33717	3.37%
Mediterranean Ave	PROPERTY	22822	2.28%
Community Chest	CHEST	19598	1.96%
Baltic Ave	PROPERTY	22921	2.29%
Income Tax	TAX	24996	2.50%
Reading Railroad	RAILROAD	32232	3.22%
Oriental Ave	PROPERTY	24319	2.43%
Chance	CHANCE	8348	0.83%
Vermont Ave	PROPERTY	24937	2.49%
Connecticut Ave	PROPERTY	24557	2.46%
Jail	JAIL	63756	6.38%
St. Charles Place	PROPERTY	29390	2.94%
Electric Company	UTILITY	30502	3.05%
States Ave	PROPERTY	24437	2.44%
Virginia Ave	PROPERTY	28036	2.80%
Pennsylvania Railroad	RAILROAD	29557	2.96%
St. James Place	PROPERTY	30589	3.06%
Community Chest	CHEST	25134	2.51%
Tennessee Ave	PROPERTY	32344	3.23%
New York Ave	PROPERTY	31962	3.20%
Free Parking	FREE PARKING	32546	3.25%
Kentucky Ave	PROPERTY	29418	2.94%
Chance	CHANCE	10806	1.08%
Indiana Ave	PROPERTY	29187	2.92%
Illinois Ave	PROPERTY	34622	3.46%
B. & O. Railroad	RAILROAD	33124	3.31%
Atlantic Ave	PROPERTY	28633	2.86%
Ventnor Ave	PROPERTY	28393	2.84%
Water Works	UTILITY	30213	3.02%
Marvin Gardens	PROPERTY	27549	2.75%
Go To Jail	GOTOJAIL	0	0.00%
Pacific Ave	PROPERTY	28564	2.86%
North Carolina Ave	PROPERTY	28141	2.81%
Community Chest	CHEST	25266	2.53%
Pennsylvania Ave	PROPERTY	26785	2.68%
Short Line Railroad	RAILROAD	26115	2.61%
Chance	CHANCE	8257	0.83%
Park Place	PROPERTY	23565	2.36%
Luxury Tax	TAX	23248	2.32%
Boardwalk	PROPERTY	28406	2.84%

Top 5 Most Visited Squares

NAME	TYPE	LAND COUNT	PERCENT
Jail	JAIL	63756	6.38%
Illinois Ave	PROPERTY	34622	3.46%
GO	GO	33717	3.37%
B. & O. Railroad	RAILROAD	33124	3.31%
Free Parking	FREE PARKING	32546	3.25%

When studying our data, we made a few keen observations of what was happening to our data across the various turn checks. At low *n* checks, such as turns = 1,000, our landing percentages appear noisy. There is no real consistency in landing percentages, and the distribution seems to be more random in nature across both Strategy A and Strategy B. As we increase *n* to larger values, we see our results stabilize and resemble the output in the tables above.

When analyzing the difference between *n* = 100,000 and *n* = 1,000,000 turns for the top 5 most landed on squares, you can start to see the data stabilize and show a clear pattern of what is happening to our player as they traverse across the board. As we can see in the data below, Strategy B at 100,000 turns is almost a spitting image of its larger counterpart. In our Strategy A, there is a bit of a variation, but only slightly, suggesting that we are seeing our data stabilize, but there is still a slight variation at this stage of our turns check.

Strategy A | Sim 10 | Turns 100000

Top 5 Most Visited Squares

NAME	TYPE	LAND COUNT	PERCENT
Jail	JAIL	6912	6.91%
Illinois Ave	PROPERTY	3790	3.79%
B. & O. Railroad	RAILROAD	3699	3.70%
GO	GO	3682	3.68%
Tennessee Ave	PROPERTY	3579	3.58%

Strategy B | Sim 10 | Turns 100000

Top 5 Most Visited Squares

NAME	TYPE	LAND COUNT	PERCENT
Jail	JAIL	6332	6.33%
Illinois Ave	PROPERTY	3508	3.51%
GO	GO	3370	3.37%
B. & O. Railroad	RAILROAD	3255	3.26%
Free Parking	FREE PARKING	3249	3.25%

Based on this data, we found that there does seem to be signs of convergence as our dataset grows. We were able to judge this based on the landing percentages for the top 5 most landed on spots. In our output to the console, we used a margin of error of 0.9% to identify convergence. We used this percentage as it seemed to encapsulate a similar margin of error

found in other Monte Carlo simulations of Monopoly. With our multiple simulated runs, we never went outside of that threshold, meaning as our number grows, our results would likely become even more stable and present more stabilization across our landing percentages.

Analysis

Once we were able to look into our results a bit further, we wanted to see how our results from Strategy A compared to Strategy B. At lower turn checks, such as n = 1000, we noticed there was a lot more randomness between the two strategies. Based on our data, we would believe these landing counts were largely affected by the change in jail behavior as well as our card mechanics.

Due to the law of large numbers, we can also assume this to be true, as those landing counts started to stabilize to their true probabilities. The law of large numbers suggests that as n increases, the random fluctuation in data decreases and the data starts to stabilize. We can see that in our results, at lower n values, there is high variability likely due to our movement mechanics, but as n increases, these effects will start to diminish.

When studying our data for Strategy A, we noticed that many of these runs tend to hover around a few particular spaces. These spaces seemed to coincide with our jail space, go, railroad, and utility spaces. This would make sense as these are the areas that players are sent to from our deck and dice mechanics.

For Strategy A, jail was consistently number one on each list. Go was the next most consistent space, hovering in the 3rd place spot. For the other three top positions, we saw more variety. We often saw Illinois Avenue move around from the 2nd position to not even being included in the top 5 listing. We suspect this has to do with our jail strategies. Since the player is not held in jail for as long in Strategy A, their probability for properties around the jail level is more smoothing the probabilities in these areas. This suggests that there were times in our simulation runs when there were likely higher chances our player visited Illinois Ave more due to our Chance and Chest cards than because of the jail.

As the turns increased for Strategy A, we started to see that at about 100,000 turns, our results started to more closely resemble the results at 1 million turns. This was determined by looking at our landing percentages. Though the positions were often flipped for 4th and 5th place, the percentage was fractions of a difference and was consistent in almost every single simulation run.

When we chose to compare our findings from Strategy A to Strategy B, we noticed that the biggest differences for smaller n values are that Strategy B landings clustered more heavily for the properties around the jail. As our n increased, we noticed that these effects started to smooth out and resembled similar results to our higher n values in Strategy A. Something that we found interesting is that even though our top 5 squares were very similar, they still showed a

higher percentage of lands for Strategy A. We think this is due to Strategy A being more easily affected by the card and jail mechanics for movement, suggesting that Strategy B reduces those effects slightly and increases the landing percentages more evenly for the spaces around the jail.

If we chose to run these same simulations with a larger group of players, we would suspect that we would retain similar results. At smaller *n* values, we would see a lot of variety, but as it were to increase, we would see our percentages stabilize and likely represent similar findings to those listed in our turns at 1 million results listed above. This is due to the fact that having multiple players doesn't affect the actual outcome of the data. Each player is independent in our simulation, and if we were to only focus on movement, then their landing percentages should not vary. If we were to account for other gameplay mechanics, this is when our data would be skewed due to the nature of bankruptcy, etc.

Conclusion

In conclusion, this project demonstrated how simulations can be used to approximate real world probabilities of complex systems such as Monopoly. By running a large number of turns, we are able to see how our data stabilizes and starts to show a clear pattern of the most probable landing spaces in our simulation. As we increased our turn count, our accuracy also increased, showing the law of large numbers and that empirical data will converge and get more stable over time. Overall, this project allowed our team to see long term effects of game mechanics on landing probabilities and showed us that sometimes it is ok to simplify mechanics to show a larger story behind the data.

Lessons Learned

The biggest lesson our team learned when implementing this project is the importance of organization and proper implementation of game design. There were a few times during implementation when we ran into basic bugs that would cause our output to create vastly wrong landing percentages. Even though we had stripped down the game mechanics and removed things like money, property, etc. we still saw that the littlest errors in logic could change everything. Debugging played a large part in getting our percentages to be more stable. This project helped to reinforce the importance of making sure we carefully implement and test each portion of our code early and often.

Works Cited

Arneson, E. (2024, September 9). 10 Most Visited Monopoly Properties. The Spruce Crafts. Retrieved April 24, 2026, from https://www.thesprucecrafts.com/most-visited-monopoly-properties-411880

Jenni. (2023, April 6). List of all Monopoly Chance & Community Chest Cards (Including New Cards!). Monopoly Land. https://www.monopolyland.com/list-monopoly-chance-community-chest-card/

Limited, A. (n.d.). *Monopoly*. Alamy Images. https://www.alamy.com/stock-photo-monopoly-26595286.html?imageid=36B2A12-64F3-4FA3-9D67-DE812B249BF9&p=4842&pn=1&searchId=d83a67e27f0f11519ed7c9456d909c1&searchtype=0

Oracle Corporation. (2025). *Java Platform, Standard Edition (JDK) 21 API specification*. https://docs.oracle.com/en/java/javase/21/docs/api/index.html

Pratt, M. (n.d.). *Monopoly Rules*. Asmodee UK. https://www.asmodee.co.uk/blogs/news/monopoly-rules

Sandy Irani, A. B. (2025). Discrete Structures. In *Discrete Mathematics.* ZyBooks.